Documento di Design

GiaLoSai

Gestione Logistica e Stoccaggio

Gruppo 11

Corso di Ingegneria del Software

Anno Accademico 2025–2026

`Versione v2.0.0`

Università Degli Studi Di Perugia — Dipartimento Di Matematica E Informatica

Contents

1 Contesto, obiettivi e non obiettivi **2**
1.1 Obiettivi . . . 2
1.2 Non obiettivi . . . 2

2 Proposta architetturale **2**
2.1 Responsabilità dei layer . . . 3
2.2 Pattern e principi effettivamente applicati . . . 4
2.3 Struttura dei package . . . 5
2.3.1 Server . . . 5
2.3.2 Client . . . 7
2.4 Persistenza, sicurezza e coerenza . . . 7
2.4.1 Database . . . 7
2.4.2 Transazioni . . . 8
2.4.3 Autenticazione . . . 9
2.5 Superficie API REST . . . 9
2.5.1 Autenticazione . . . 10
2.5.2 Prodotti . . . 10
2.5.3 Ordini . . . 10
2.5.4 Categorie e utenti . . . 11
2.6 Architettura client TUI . . . 11
2.6.1 Navigazione . . . 11
2.6.2 Gerarchia delle pagine . . . 12
2.7 Topologia di deploy . . . 13
2.7.1 Servizi . . . 13
2.7.2 Dockerfile multi-stage . . . 13
2.7.3 Avvio . . . 13
2.8 Flussi dati principali . . . 14
2.8.1 Login . . . 14
2.8.2 Ordine di uscita con generazione DDT . . . 14
2.8.3 Gestione resi . . . 14
2.8.4 Fatture e note di credito . . . 14
2.9 Strategia di testing . . . 14
2.9.1 Livelli di testing . . . 16

3 Alternative considerate **16**

4 Conseguenze e limiti **17**

1 Contesto, obiettivi e non obiettivi

GiaLoSai è un sistema client–server per la gestione logistica interna di un'azienda: prodotti, categorie, giacenze, ordini di entrata e uscita, DDT, fatture, note di credito, resi e utenti con ruoli differenziati. Il backend è scritto in Go, espone API REST (Representational State Transfer) e persiste su MySQL 8 tramite Docker Compose. Il client è una TUI (Text-based User Interface) scritta in Go con il framework Bubble Tea.

Nota: Il progetto segue le best practice dei moderni strumenti CLI: come `docker` e `kubectl`, GiaLoSai separa il client leggero (TUI) dal backend, comunicando tramite API REST con autenticazione JWT (JSON Web Token). Il pattern è lo stesso utilizzato da `gh` (GitHub CLI) e `aws` CLI.

1.1 Obiettivi

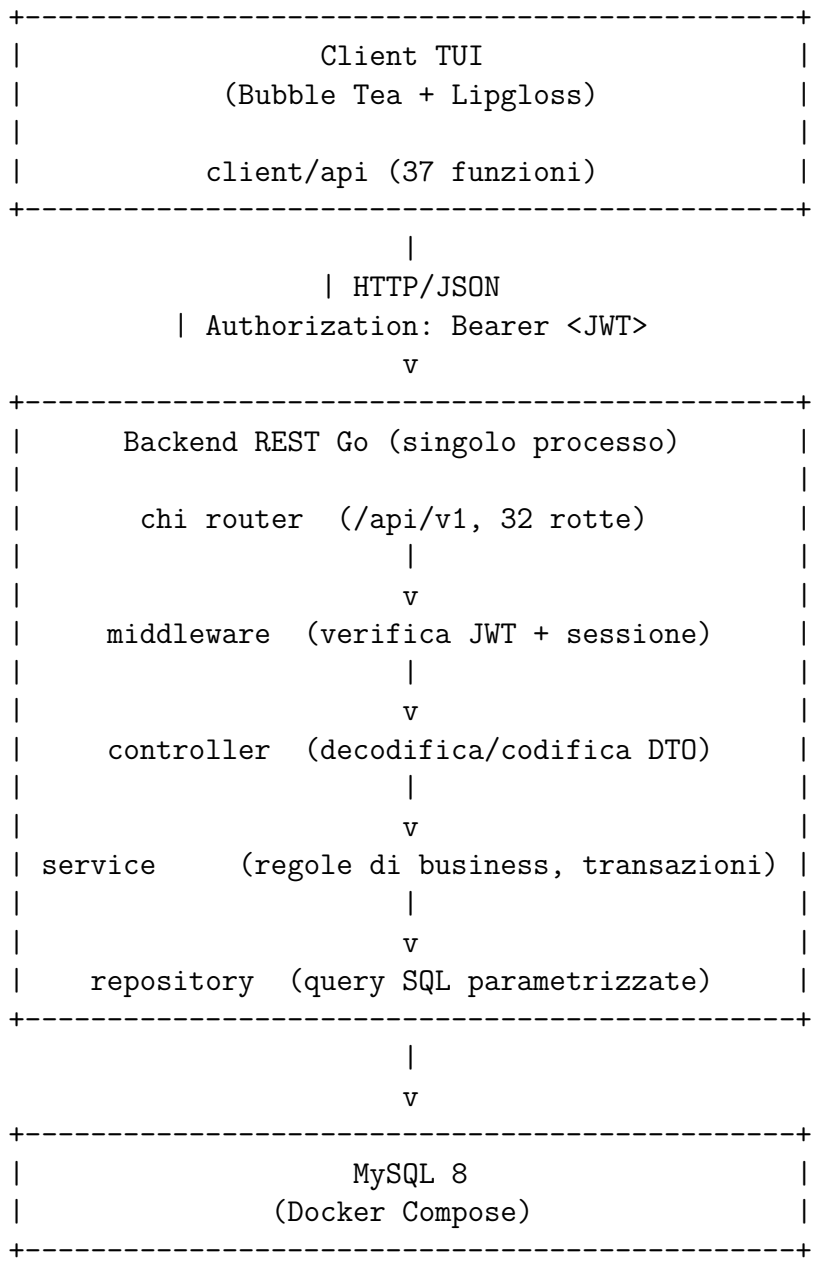
- Gestire l'intero ciclo della merce: dall'anagrafica prodotti fino a DDT, fatture, note di credito e resi.
- Fornire tre ruoli (magazziniere, contabile, admin) con funzionalità distinte.
- Garantire atomicità delle operazioni multi-entità tramite transazioni MySQL.
- Raggiungere un livello documentale e testuale adeguato a un progetto universitario.

1.2 Non obiettivi

- Canale e-commerce o raccolta ordini dal cliente finale.
- Generazione materiale di PDF o CSV: i percorsi sono registrati nei modelli ma non esiste un generatore.
- Microservizi, orchestratori (Kubernetes) o distribuzione cloud.
- Autenticazione OAuth2/OIDC o federazione SSO.

2 Proposta architetturale

Il sistema combina gli stili *client–server*, *layered* e *repository*. Il client TUI in Go è un processo separato dal backend e comunica con esso in HTTP/JSON. Il backend Go centralizza le regole applicative e accede a un repository dati MySQL condiviso. Non è un'architettura a microservizi: i package del server sono moduli di un singolo processo.



Nota: L'architettura segue lo stesso pattern di `docker`: un binario singolo che espone API HTTP, con un client TUI separato che interagisce con esso. Il modello "backend + client separati" è lo stesso utilizzato da `portainer`, `k9s` e `lazygit`.

La soluzione risponde a tre esigenze: separare l'esperienza utente dalla logica applicativa, concentrare la persistenza in un punto controllato e mantenere le regole di dominio testabili senza richiedere sempre risorse esterne.

2.1 Responsabilità dei layer

Layer	Responsabilità	Package
Client TUI	Gestisce input, stato e rendering secondo Model–View–Update; navigazione multi-pagina con variabili di tipo "Enum" `iota`.	`client/ui/app`
API Client	Traduce le operazioni TUI in chiamate HTTP REST; conserva JWT e ruolo in memoria.	`client/api`
Router e middleware	Compone le dipendenze (`NewRouter`), espone 32 rotte `/api/v1`, separa login pubblico da rotte protette.	`server/router`, `server/middleware`
Controller	Adattatore HTTP: decodifica DTO, chiama il service, codifica la risposta JSON.	`server/{auth, user, product, category, order}`
Service	Regole di business, coordinamento transazionale, validazione.	`server/{auth, user, product, category, order}`
Repository	Incapsula query SQL parametrizzate; è l'unico layer che conosce lo schema MySQL.	`server/{auth, user, product, category, order}`
Domain	Entità, costanti, errori sentinella, validazione EAN-13, `TransactionHelper`.	`server/domain`

2.2 Pattern e principi effettivamente applicati

Il modello è un'architettura a layer, non un MVC classico: i controller sono adattatori HTTP e la view è nel processo client separato.

- **Repository pattern**: ogni package di dominio ha un repository che incapsula l'accesso SQL. I service dipendono da interfacce ristrette (non dalle implementazioni concrete), coerentemente con *Dependency Inversion* e *Interface Segregation*. Ciò consente l'uso di mock (`go-sqlmock`) nei test. Questo pattern è lo stesso utilizzato da `database/sql` in Go e da progetti come `github.com/jmoiron/sqlx`.
- **DTO (Data Transfer Object)**: le strutture di trasporto HTTP sono separate dalle entità del dominio. Ogni package ha i propri DTO (`AuthDTO`, `UserDto`,

`ProductDto`, `OrderDto`, `CategoryDto`). Il pattern è lo stesso adottato da `docker` API e `kubernetes` API.

- **Transaction-in-context**: `TransactionHelper.WithinTransaction` inserisce una transazione SQL nel `context.Context`; i repository la estraggono con `GetExecutor`. Il pattern è trasparente ai service e supporta nesting. Simile al pattern utilizzato da `sqlx` e `gorm`.
- **Model–View–Update (MVU)**: il client TUI applica il pattern Elm su ogni pagina. Ogni package pagina espone `Init()`, `Update()` e `View()` come interfaccia Bubble Tea. Lo stesso pattern è utilizzato da `lazygit`, `k9s`, `btop` e `htop`.
- **Sentinel errors**: `domain/errors.go` definisce errori sentinella (`ErrGiacenzaInsufficient`, `ErrAccountBloccato`, `ErrCredenzaliErrate`, ecc.) per il controllo degli flussi di errore nei service. Pattern standard in Go, utilizzato anche da `net/http` e `io`.

Non vengono dichiarati Strategy, Factory, Observer o Adapter come pattern di progetto, poiché il codice non contiene collaboratori intercambiabili o punti di estensione che ne giustifichino l'attribuzione.

2.3 Struttura dei package

Il progetto è organizzato in due moduli Go indipendenti (`client/go.mod` e `server/go.mod`), ciascuno con le proprie dipendenze.

2.3.1 Server

```
server/
+-- cmd/gialosai/        # Entry point (main.go)
+-- router/              # Composizione dipendenze e rotte REST
|   +-- Router.go        # NewRouter(db): 32 endpoint
|   +-- Router_test.go
|   +-- integration_test.go
+-- middleware/           # Autenticazione HTTP
|   +-- AuthMiddleware.go # Verifica JWT + sessione DB
|   +-- AuthMiddleware_test.go
+-- domain/              # Modello comune
|   +-- models.go        # 12 entita (Utente, Prodotto, OrdineUscita, DDT, ...)
|   +-- constants.go     # Ruoli, stati, soglie, prefissi documenti
|   +-- errors.go        # 10 errori sentinella
|   +-- db.go            # TransactionHelper, QueryExecutor
|   +-- validation.go    # IsValidEAN13
|   +-- reports.go       # InventoryItem, InventoryReport
+-- auth/                # Login, JWT, sessioni
|   +-- AuthController.go
|   +-- AuthService.go
|   +-- AuthJWT.go       # HS256, hash SHA-256 del token
|   +-- AuthPassword.go  # bcrypt (cost 10)
|   +-- AuthDTO.go
|   +-- SessionRepository.go
+-- user/                # CRUD utenti, blocco account
```

```
+-- product/              # Catalogo prodotti, giacenze, report
+-- category/             # CRUD categorie
+-- order/                # Ordini, DDT, fatture, resi
```

Lo schema DDL (`docker/mysql/init.sql`) e i dati seed (`docker/mysql/prova.sql`) risiedono nella cartella `docker/mysql/`, non nel modulo server.

2.3.2 Client

```
client/
+-- cmd/gialosai/          # Entry point client
+-- api/                   # Client HTTP
|   +-- client.go          # APIClient, Login()
|   +-- products.go        # GetCatalogo, CreaProdotto, GetReportGiacenze, ...
|   +-- orders.go          # GetOrdini, CreaOrdineUscita, GetDDT, ...
|   +-- users.go           # CreateUser, ToggleUserBlock, ...
|   +-- model.go           # DTO lato client (Prodotto, Ordine, Fattura, ...)
+-- ui/
|   +-- app/               # Modello principale (mainLayout)
|   |   +-- app.go         # Update() con switch navigazione
|   |   +-- model.go       # Enum paginaAttiva, userRole con iota
|   +-- styles/            # Stili condivisi (bordi, colori, dimensioni)
|   +-- components/        # Header, footer riutilizzabili
|   +-- tabnav/            # Navigazione a tab con hitbox click
|   +-- pages/
|       +-- login/
|       +-- magazzino/     # ordinientrata, ordiniuscita, giacenze,
|       |                  # scanner, resi
|       +-- contabilita/   # ordini, fatture, report, catalogo
|       +-- admin/         # dashboard, users
```

Nota: La struttura del client segue lo stesso pattern di `lazygit` e `k9s`: un modello principale che ospita sub-model per ogni pagina, con navigazione a enum e stili condivisi. Il pattern è identico a quello utilizzato da `bubbletea` nella documentazione ufficiale.

Ogni package pagina (`login/`, `giacenze/`, `fatture/`, ...) dichiara i propri metodi Bubble Tea (`Init`, `Update`, `View`) e un costruttore `New()`.

2.4 Persistenza, sicurezza e coerenza

2.4.1 Database

MySQL 8 è eseguito tramite Docker Compose. Lo schema (`docker/mysql/init.sql`) definisce 12 tabelle con engine InnoDB, collazione `utf8mb4_unicode_ci` e chiavi esterne.

Tabella	Descrizione
`Utenti`	Account con ruolo (`magazziniere`, `contabile`, `admin`), tentativi falliti e flag blocco.
`Sessioni`	Whitelist JWT: hash SHA-256 del token, scadenza, FK a `Utenti`.

Tabella	Descrizione
`Categorie`	Anagrafica categorie prodotto (nome univoco, descrizione).
`Prodotti`	Catalogo con codice, barcode EAN-13, prezzi di acquisto/vendita, soglia minima, FK a `Categorie`.
`Giacenze`	Quantità disponibile per prodotto (1:1 con `Prodotti`, cascade delete).
`OrdiniEntrata`	Ordini fornitore con numero progressivo, stato e note.
`RigheOrdineEntrata`	Righe ordine entrata: prodotto, quantità, prezzo unitario.
`OrdiniUscita`	Ordini cliente con stato (`confermato`, `evaso`, `annullato`) e FK opzionale a DDT.
`RigheOrdineUscita`	Righe ordine uscita: prodotto, quantità, prezzo unitario.
`DDT`	Documento di trasporto collegato a un ordine uscita (1:1).
`Fatture`	Fatture di vendita e note di credito (`tipo`: `fattura` o `nota_credito`), con imponibile, IVA e totale.
`Resi`	Resi merce con stato (`da_verificare`, `accettato`, `rifiutato`) e motivo.

Le chiavi esterne garantiscono l'integrità referenziale: ogni prodotto ha una giacenza, le righe dipendono dagli ordini (cascade delete), un DDT è associato a un solo ordine di uscita, e un ordine creato da un utente non può essere eliminato se l'utente viene rimosso (`RESTRICT`).

2.4.2 Transazioni

`TransactionHelper` (`domain/db.go`) apre una transazione e la propaga nel `context.Context`. I repository estraggono l'executor con `GetExecutor(ctx, r.DB)`, che restituisce la transazione se presente nel contesto, altrimenti la connessione diretta. Questo confine atomico protegge:

- **Ordini di entrata**: inserimento righe e incremento giacenze nello stesso `BEGIN/COMMIT`.
- **Ordini di uscita**: verifica disponibilità, decremento giacenze e registrazione ordine. Un errore di giacenza insufficiente causa `ROLLBACK`.
- **Login**: read-modify-write atomico dei tentativi falliti e del flag blocco.
- **Resi**: aggiornamento stato e ripristino/reintegro giacenza in transazione.

Nota: Il pattern di transazione in contesto è lo stesso utilizzato da `database/sql` di Go e da ORM come `gorm`. La propagazione via `context.Context` è la best practice raccomandata dal team di Go per la gestione delle transazioni.

2.4.3 Autenticazione

Il flusso di autenticazione è il seguente:

1. Il client invia `POST /api/v1/login` con username e password.
2. Il server verifica le credenziali con `bcrypt.CompareHashAndPassword`.
3. Se valide, genera un JWT HS256 (`golang-jwt/jwt/v5`) con claim `user_id`, `username`, `role` e scadenza 24h.
4. Il token viene hashato con SHA-256 e salvato nella tabella `Sessioni` (mai in chiaro).
5. Il client conserva il JWT e lo invia come `Authorization: Bearer <token>` a ogni richiesta protetta.
6. Il middleware verifica: (a) firma JWT e scadenza, (b) presenza della sessione hashata nel DB, (c) inietta `userID` nel contesto.

Nota: L'autenticazione JWT con whitelist è lo stesso approccio utilizzato da `docker` (con token di accesso) e `kubectl` (con certificati e token). Il pattern di hash SHA-256 del token è una best practice per evitare che token compromessi siano riutilizzabili.

Dopo tre tentativi di accesso falliti consecutivi, l'account viene bloccato (`RF-02`). L'account con username `admin` è esente dal blocco. Il ripristino avviene solo tramite un altro admin che sblocca l'account e imposta una nuova password.

2.5 Superficie API REST

Il server espone 32 endpoint sotto il prefisso `/api/v1`. Tutte le rotte richiedono autenticazione tranne il login.

Nota: Le API REST seguono le stesse convenzioni di `docker` API (`/containers/create`, `/images/json`) e `kubernetes` API (`/api/v1/namespaces`). Il pattern CRUD con endpoint nominativi è lo standard per API REST progettate per client TUI.

2.5.1 Autenticazione

Metodo	Percorso	Operazione
POST	/api/v1/login	Autentica e restituisce token JWT e ruolo.

2.5.2 Prodotti

Metodo	Percorso	Operazione
GET	/api/v1/products/	Elenca tutti i prodotti.
POST	/api/v1/products/	Crea un nuovo prodotto.
GET	/api/v1/products/inventory-report	Restituisce il report inventario.
PUT	/api/v1/products/{id}/price	Aggiorna il prezzo di vendita.
DELETE	/api/v1/products/{id}	Elimina un prodotto.
GET	/api/v1/products/barcode/{barcode}	Cerca prodotto per barcode EAN-13.

2.5.3 Ordini

Metodo	Percorso	Operazione
GET	/api/v1/orders/	Ordini di uscita.
GET	/api/v1/orders/uscita	Ordini di uscita (alias).
GET	/api/v1/orders/entrata	Ordini di entrata.
POST	/api/v1/orders/crea-uscita	Crea ordine di uscita.
POST	/api/v1/orders/crea-entrata	Crea ordine di entrata.
PUT	/api/v1/orders/update-ordine-uscita/{id}	Aggiorna ordine uscita.
DELETE	/api/v1/orders/delete-entrata/{id}	Elimina ordine entrata.
DELETE	/api/v1/orders/delete-uscita/{id}	Elimina ordine uscita.
POST	/api/v1/orders/{id}/create-ddt	Genera DDT per ordine confermato, portandolo a evaso.

Metodo	Percorso	Operazione
POST	/api/v1/orders/create-fattura/{id}	Genera fattura di vendita.
DELETE	/api/v1/orders/delete-fattura/{id}	Elimina fattura.
POST	/api/v1/orders/create-credit-notes/{id}	Genera nota di credito.
GET	/api/v1/orders/invoices	Elenca fatture.
GET	/api/v1/orders/credit-notes	Elenca note di credito.
GET	/api/v1/orders/resi	Elenca resi.
POST	/api/v1/orders/crea-reso	Crea un reso.
PUT	/api/v1/orders/update-resi	Aggiorna stato reso.
DELETE	/api/v1/orders/delete-reso/{id}	Elimina un reso.

2.5.4 Categorie e utenti

Metodo	Percorso	Operazione
GET	/api/v1/categories/	Elenca categorie.
POST	/api/v1/categories/	Crea categoria.
DELETE	/api/v1/categories/{id}	Elimina categoria.
POST	/api/v1/users/	Crea utente.
GET	/api/v1/users/{username}	Cerca utente per username.
PUT	/api/v1/users/{username}	Aggiorna utente.
DELETE	/api/v1/users/{username}	Elimina utente.

2.6 Architettura client TUI

Il client è una TUI Go basata su Bubble Tea v2 (architettura Model–View–Update) con Lipgloss v2 per lo stile. Il modello principale `mainLayout` (`client/ui/app/`) ospita tutti i sub-model e gestisce la navigazione.

Nota: Bubble Tea è il framework TUI più utilizzato nell'ecosistema Go, con progetti come `lazygit` (25k+ stelle), `glow` (rendering markdown), `soft-serve` (server Git TUI) e `charm` (la stessa azienda che lo sviluppa). Il pattern MVU è lo stesso di `Elm` e `Redux`, reso popolare da applicazioni web reattive.

2.6.1 Navigazione

La navigazione è controllata da un enum `paginaAttiva` definito con `iota`:

```
const (
    LOGIN     paginaAttiva = iota  // 0
    MAGMENU                        // 1 - Menu magazzino
    CONTMENU                       // 2 - Menu contabilità
    ADMINMENU                      // 3 - Menu admin
)
```

Ogni menu espone le proprie sottopagine. Il metodo `Update()` del modello principale esegue uno `switch` sul valore corrente di `paginaAttiva` per instradare i messaggi alla sottopagina attiva. La navigazione avviene tramite scorciatoie da tastiera (`ctrl+F1`, `ctrl+F2`, ...) e `ctrl+L` per il logout.

Nota: Il sistema di navigazione a enum è lo stesso utilizzato da `lazygit`, `k9s` e `btop`. La navigazione a scorciatoie da tastiera è una best practice per TUI, garantendo efficienza e accessibilità.

2.6.2 Gerarchia delle pagine

Pagina	Package	Ruolo
Login	`pages/login/`	Tutti
Menu Magazzino	`pages/magazzino/`	Magazziniere
Giacenze	`pages/magazzino/giacenze/`	Magazziniere
Ordini Entrata	`pages/magazzino/ordinientrata/`	Magazziniere
Ordini Uscita	`pages/magazzino/ordiniuscita/`	Magazziniere
Scanner	`pages/magazzino/scanner/`	Magazziniere
Resi	`pages/magazzino/resi/`	Magazziniere
Menu Contabilità	`pages/contabilita/`	Contabile
Ordini	`pages/contabilita/ordini/`	Contabile
Fatture	`pages/contabilita/fatture/`	Contabile
Report	`pages/contabilita/report/`	Contabile
Catalogo	`pages/contabilita/catalogo/`	Contabile
Menu Admin	`pages/admin/`	Admin
Dashboard	`pages/admin/dashboard/`	Admin
Utenti	`pages/admin/users/`	Admin
Accesso esteso	`pages/magazzino/`, `pages/contabilita/`	Admin

Oltre alle proprie pagine, l'Admin ha accesso anche a tutte le pagine e ai servizi riservati a Magazziniere e Contabile (giacenze, ordini entrata/uscita, resi, scanner, catalogo, fatture, report): non è quindi limitato alla sola sezione `pages/admin/`.

Il client API (`client/api/`) espone 37 funzioni esportate che coprono login, catalogo, ordini, fatture, resi e gestione utenti. Ogni funzione effettua una richiesta HTTP, deserializza la risposta JSON e restituisce il DTO lato client.

2.7 Topologia di deploy

Il sistema è containerizzato con Docker Compose. Due servizi (`db` e `app`) sono definiti in `docker/docker-compose.yml`.

> **Nota:** L'approccio Docker segue le best practice di `docker` e `docker-compose`: un file YAML che definisce i servizi, le dipendenze e i volumi. Il pattern multi-stage build è lo stesso utilizzato da `golang` official image e da progetti come `traefik`.

2.7.1 Servizi

Servizio	Immagine	Configurazione
`db`	`mysql:8.0`	Porta host 3307, volume `mysql_data`, healthcheck `mysqladmin ping`, init scripts da `docker/mysql/` (schema + seed).
`app`	Build locale (Dockerfile multi-stage)	Porta 8080, dipende da `db` con `condition: service_healthy`, volume `../output:/app/output`.

2.7.2 Dockerfile multi-stage

1. **Builder**: `golang:1.25-alpine` compila il server con `CGO_ENABLED=0`, `-trimpath`, `-ldflags="-s -w"`.
2. **Runtime**: `gcr.io/distroless/static-debian12:nonroot` esegue il binario come utente `nonroot:nonroot`, minimizzando la superficie di attacco.

2.7.3 Avvio

```
# Avvio completo (DB + server)
docker compose -f docker/docker-compose.yml up --build

# Client TUI (in un secondo terminale)
cd client && go run .
```

Compose avvia MySQL con healthcheck e il backend sulla porta 8080. Il client si connette a `http://localhost:8080`. Il volume `mysql_data` conserva i dati tra i restart; eseguire `docker compose down -v` per ripartire da un database vuoto.

2.8 Flussi dati principali

2.8.1 Login

1. Client invia `POST /api/v1/login` con {"username", "password"}.
2. `AuthService_Login` estrae l'utente dal DB, verifica blocco e password bcrypt.
3. Se ok: genera JWT (HS256, scadenza 24h), hash SHA-256 del token, salva in `Sessioni`, resetta tentativi falliti.
4. Se password errata: incrementa tentativi; se ≥ 3 imposta `bloccato = true`.
5. Risposta: {"token": "...", "role": "magazziniere"}.

2.8.2 Ordine di uscita con generazione DDT

1. Magazziniere compila l'ordine nella TUI e invia `POST /api/v1/orders/crea-uscita`.
2. `OrderService` apre una transazione con `WithinTransaction`.
3. Per ogni riga: verifica che `quantita` $\leq$ giacenza disponibile. Se insufficiente, restituisce `GiacenzaInsufficienteError` e la transazione esegue `ROLLBACK`.
4. Se tutte le quantità sono sufficienti: decrementa le giacenze, registra l'ordine con stato `confermato`, commit.
5. Il magazziniere genera il DDT con `POST /api/v1/orders/{id}/create-ddt`: il service assegna una numerazione annuale progressiva, crea il record DDT e aggiorna lo stato dell'ordine a `evaso`.

2.8.3 Gestione resi

1. Il reso viene creato con stato `da_verificare`.
2. alla verifica (`PUT /api/v1/orders/update-resi`), lo stato passa a `accettato` o `rifiutato`.

2.8.4 Fatture e note di credito

Fatture e note di credito vengono emesse per ordini di uscita nello stato `evaso` e il loro importo è calcolato a partire dalle righe dell'ordine applicando l'IVA al **22%**:

1. `imponibile` $= \sum_{riga}$ (`quantita` $\times$ `prezzo_unitario`);
2. `IVA` $=$ `imponibile` $\times 0.22$;
3. `totale` $=$ `imponibile` $+$ `IVA`.

Il valore mostrato nella TUI per una fattura o nota di credito è il **totale**, cioè prezzo + IVA al 22% (`imponibile` *e* `IVA` restano comunque memorizzati e restituiti dal backend).

2.9 Strategia di testing

Il progetto comprende 43 file di test distribuiti tra server (21) e client (22), eseguiti con `go test -v ./...` tramite `make test`. Tra questi, la cartella `server/router/` ospita una suite di *test di integrazione end-to-end* (`integration_test.go`) che esercita l'intera catena router–middleware–controller–service–repository con database mockato: un livello distinto dai test di unità di package.

Nota: L'approccio al testing segue le best practice di `go test`: test di unità con mock, test di integrazione con database transazionali, e test TUI con il framework `bubbletea/testing`. Questo è lo stesso approccio utilizzato da `docker`, `kubernetes` e `helm`.

2.9.1 Livelli di testing

Livello	Strumenti	Copertura
Unit test domain	`go test`	Validazione EAN-13, errori sentinella, costanti, transazioni.
Unit test service	`testify` + `go-sqlmock`	Login, CRUD utenti/prodotti/categorie, ordini, DDT, fatture, resi.
Unit test repository	`go-sqlmock`	Query SQL parametrizzate, gestione errori DB.
Unit test controller	`httptest`	Decodifica DTO, risposte HTTP, gestione errori.
Test middleware	`httptest`	Verifica JWT, sessione, iniezione contesto.
Test di integrazione router	`httptest` + `go-sqlmock`	Suite end-to-end (`integration_test.go`, 17 casi): richiesta HTTP grezza → router → middleware → controller → service → repository → DB mockato, verificando il contratto frontend/backend (path, metodo, status code, body JSON).
Test client API	`httptest`	Deserializzazione risposte, gestione errori HTTP.
Test TUI	`bubbletea/testing`	Init, Update, View di ogni pagina; hitbox click; scroll.

3 Alternative considerate

Le alternative progettuali principali sono state valutate e scartate per i motivi seguenti.

- **GUI o applicazione web al posto della TUI.** Un'interfaccia grafica o web avrebbe richiesto più lavoro di configurazione e distribuzione senza aggiungere capacità richieste dai vincoli del progetto; la TUI resta eseguibile da terminale e separata dal backend tramite API HTTP. Il successo di TUI come `htop`, `lazygit`, `k9s` e `btop` dimostra che un'interfaccia testuale ben progettata può essere più efficiente di una GUI per operazioni amministrative.

- **SQLite al posto di MySQL.** Un database embedded semplifica l'avvio locale, ma il sistema richiede un repository condiviso multiutente con vincoli relazionali e operazioni multi-entità; MySQL 8, gestito con Docker Compose, centralizza dati e regole in un punto controllato.
- **Architettura a microservizi.** La separazione in servizi indipendenti avrebbe introdotto complessità di comunicazione e di deploy non giustificata dalle dimensioni del sistema; si è preferito un processo monolitico a layer con confini espliciti tra router, controller, service, repository e domain.
- **DAO generico al posto del Repository pattern.** Un accesso ai dati di tipo DAO sarebbe stato più generico ma avrebbe accoppiato i service al dialetto SQL; i repository, associati alle aree di dominio, espongono interfacce ristrette coerenti con Dependency Inversion e facilitano i test con mock.
- **ORM (GORM) al posto di `database/sql`.** Un ORM avrebbe ridotto il boilerplate delle query ma avrebbe introdotto un livello di astrazione opaco, harder da controllare per operazioni transazionali complesse e meno trasparente nel debugging.
- **Sessioni classiche (cookie) al posto di JWT.** Le sessioni con cookie non sono adatte a un client TUI che non ha un browser; JWT con whitelist nel DB consente logout effettivo e controllo server-side della sessione. Il pattern è lo stesso utilizzato da `docker` (con token di accesso) e `kubectl` (con certificati e token).

4 Conseguenze e limiti

La separazione aumenta coesione e testabilità, ma introduce dipendenza dalla rete locale e dalla disponibilità di MySQL. Il repository centralizzato semplifica la consistenza dei dati, ma costituisce un punto operativo comune da gestire con backup e monitoraggio.

L'integrazione finale (v2.0.0) ha allineato client e server: le principali funzionalità sono esposte via API e raggiungibili dalla navigazione TUI. I limiti residui riguardano:

- **Autorizzazione RBAC**: il ruolo è presente nel token JWT ma non è ancora verificato dal router sulle singole rotte. Qualsiasi utente autenticato può accedere a qualunque endpoint protetto.
- **Generazione documenti**: i percorsi PDF sono registrati nei modelli (`DDT.PathPDF`, `Fattura.PathPDF`) ma il codice non contiene un generatore di PDF o CSV.
- **Bootstrap primo utente**: l'ambiente Compose fornisce utenti di prueba via seed script; avviando il server sull'host senza Compose, il primo utente deve essere creato manualmente nel database con password bcrypt.